

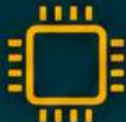
EMBEDDED SYSTEMS

EMBEDDED COMMUNICATION PROTOCOLS

CONNECTING DEVICES. ENABLING INTELLIGENCE.



UART



SPI



I²C



CAN



MULTIPLE PROTOCOLS
One goal – Reliable Communication



FASTER. SMARTER. STRONGER.
Built for Performance



REAL-WORLD APPLICATIONS
From Devices to Systems



EMBEDDED. CONNECTED. LIMITLESS.
The Future is Communication



WHAT ARE EMBEDDED PROTOCOLS?

Embedded protocols are standard rules and formats that define how microcontrollers, sensors, ICs and modules exchange data with each other.

WHY ARE THEY IMPORTANT?



CONNECT

Enable devices to communicate and work together.



EFFICIENT

Transfer data quickly and reliably.



RELIABLE

Ensure error detection and data integrity.



COMPATIBLE

Standard protocols make systems scalable and interoperable.



INDUSTRY READY

Used in automotive, consumer, industrial & IoT applications.

WHAT WE WILL COVER

UART



Asynchronous serial communication using TX & RX lines.



SPI



Synchronous communication using MOSI, MISO, SCLK & CS lines.



I²C



Two-wire communication using SDA & SCL lines with addressing.



CAN



Robust multi-node communication using CANH & CANL lines.



These protocols are the foundation of embedded systems. Master them and you can connect anything, anywhere!

UART

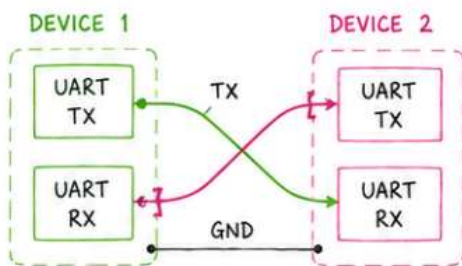
(Universal Asynchronous Receiver Transmitter)

UART is an asynchronous serial communication protocol that uses two wires : **TX (Transmit)** and **RX (Receive)**.

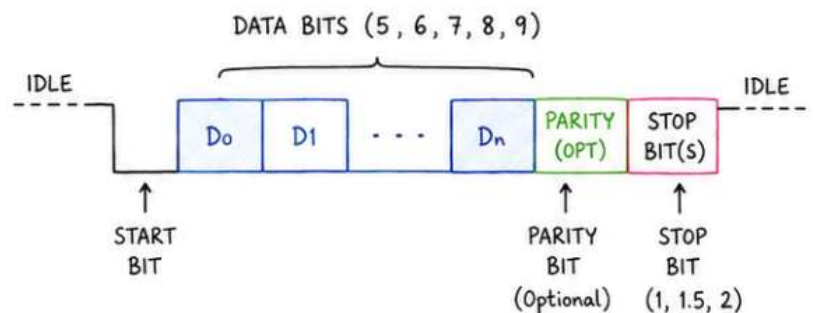
KEY POINTS

- Asynchronous
- Full-duplex
- Uses TX and RX lines
- Common GND
- Simple & widely used

UART BLOCK DIAGRAM



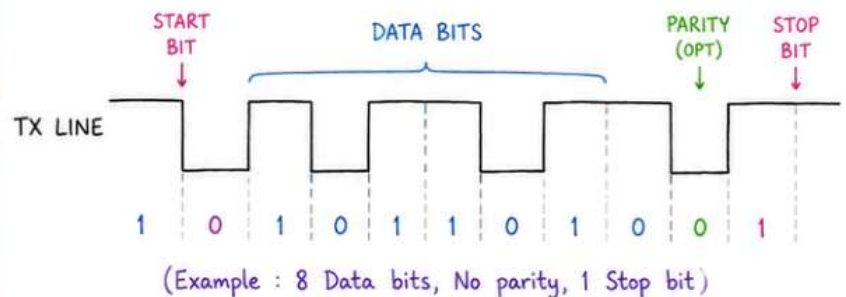
UART FRAME FORMAT



HOW UART WORKS ?

- Data is sent frame by frame.
- Each frame has Start bit, Data bits, (Optional Parity) and Stop bit(s).
- Receiver samples data bits in the middle of each bit period.
- Baud rate must be same on both transmitter and receiver.

UART SIGNAL WAVEFORM (EXAMPLE)



COMMON UART SETTINGS

- ☆ Baud Rate : 9600, 19200, 115200, etc.
- ☆ Data Bits : 5, 6, 7, 8, 9
- ☆ Parity : None, Even, Odd
- ☆ Stop Bits : 1, 1.5, 2
- ☆ Flow Control : None, RTS/CTS (optional)

APPLICATIONS



ADVANTAGES

- ✓ Very simple & easy to implement
- ✓ Requires only 2 wires (TX, RX)
- ✓ Widely supported
- ✓ Good for long distance (with proper level shifting)

LIMITATIONS

- ✗ Lower data rate compared to SPI / I²C
- ✗ No addressing (point to point communication)

Remember!

TX of one device connects to RX of the other device.

SPI

(Serial Peripheral Interface)

What is SPI?

SPI is a **synchronous** serial communication protocol used for short distance communication between a **master** and one or more **slave** devices.



SPI PINS (4-WIRE)

- MOSI** → Master Out Slave In (Data from Master to Slave)
- MISO** → Master In Slave Out (Data from Slave to Master)
- SCLK** → Serial Clock (Clock from Master)
- CS/SS** → Chip Select / Slave Select (Selects the slave device)

DATA TRANSMISSION (FULL-DUPLEX)

While Master sends data on MOSI, it also receives data on MISO at the **same time**.



FEATURES

- Synchronous communication
- Full-duplex
- High speed (faster than I²C and UART)
- Simple protocol
- No addressing (uses CS line for slave select)



ADVANTAGES

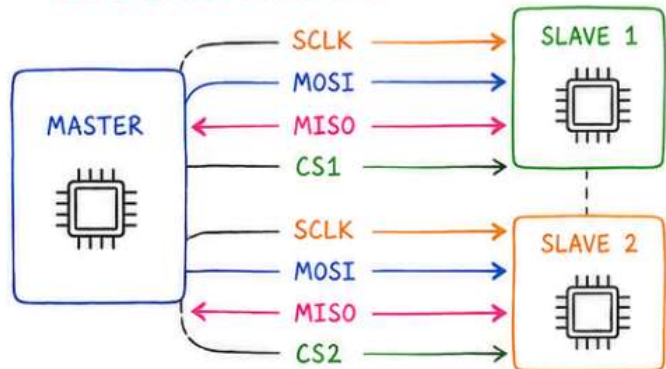
- ✓ Very fast data transfer
- ✓ Simple hardware and easy to implement
- ✓ Supports multiple slaves
- ✓ Efficient for short distance communication



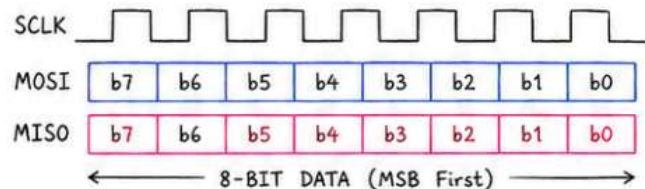
KEY POINTS

- ★ Full-duplex communication
- ★ Uses 4 main signal lines
- ★ High speed and efficient
- ★ Master controls the communication
- ★ No addressing in basic SPI (select line used)

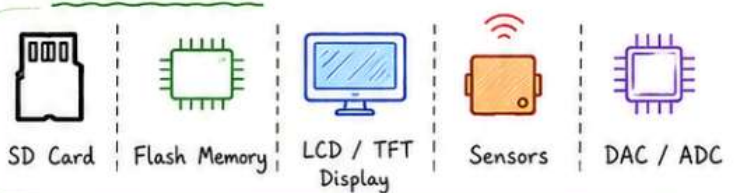
SPI BLOCK DIAGRAM



SPI FRAME FORMAT



APPLICATIONS



LIMITATIONS

- ✗ More wires (4 wires) required
- ✗ Not suitable for long distance
- ✗ No built-in error checking



QUICK TIP : Always pull CS high (inactive) when the slave is not selected. Data is transferred on clock edges (check **CPOL** & **CPHA** settings).



I²C

(Inter-Integrated Circuit)

What is I²C ?

I²C is a **synchronous**, two-wire serial communication protocol used to connect a **master** device with one or more **slave** devices.



KEY POINTS

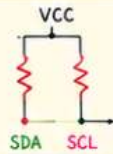
- Uses only 2 wires : **SDA** (Data) and **SCL** (Clock)
- Supports multiple slave devices on the same bus
- Master initiates communication
- Each slave has a unique 7-bit (or 10-bit) address
- Data transfer with Acknowledge (**ACK** / **NACK**)
- Half-duplex communication

I²C PINS

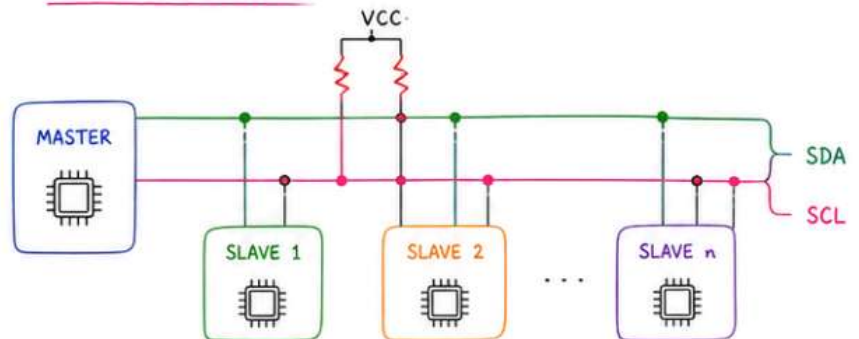
SDA → Serial Data Line (Bi-directional)
SCL → Serial Clock Line (From Master)

★ Note :

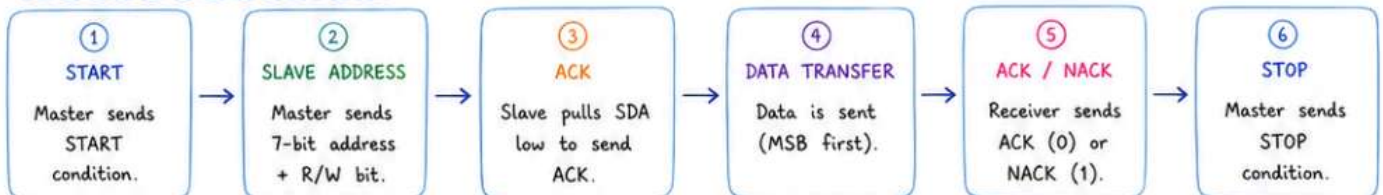
Both lines are open-drain and require pull-up resistors (4.7kΩ to 10kΩ) to VCC.



I²C BUS DIAGRAM



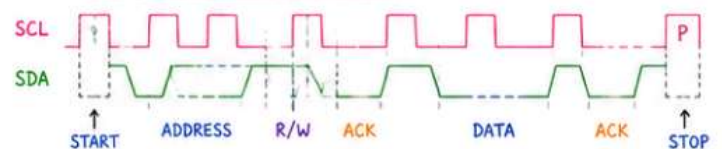
I²C COMMUNICATION STEPS



I²C FRAME FORMAT



I²C WAVEFORM (EXAMPLE)



APPLICATIONS

- EEPROM
- RTC (Real Time Clock)
- LCD Displays
- Sensors (Temperature, Pressure, etc.)
- Battery Management ICs



ADVANTAGES

- ✓ Uses only 2 wires
- ✓ Supports multiple slaves
- ✓ Simple and low cost
- ✓ Suitable for short distance
- ✓ Built-in ACK/NACK



LIMITATIONS

- ✗ Slower than SPI and UART
- ✗ Limited distance (typically < 1 meter)
- ✗ Not suitable for high speed data
- ✗ Address conflict if two slaves have same address



QUICK TIP

Always connect pull-up resistors to SDA and SCL. Without pull-ups, I²C communication will not work!

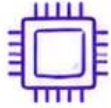


CAN

(Controller Area Network)

WHAT IS CAN ?

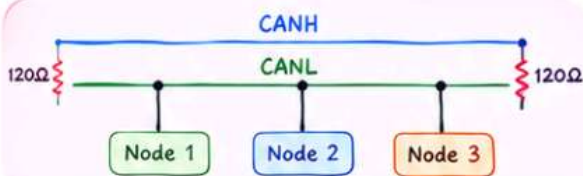
CAN is a serial communication protocol for **real-time**, **reliable** and **noise-resistant** communication in embedded systems, especially in automotive and industrial applications.



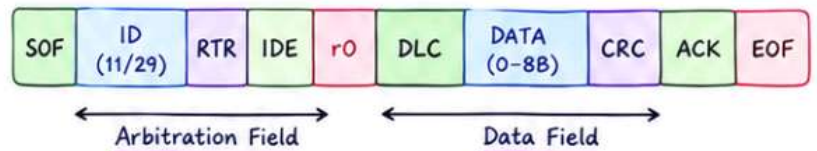
KEY FEATURES

- ✓ Multi-master, message-based protocol
- ✓ High noise immunity (differential signaling)
- ✓ Error detection and automatic retransmission
- ✓ Supports real-time communication
- ✓ Used in cars, industrial control, IoT

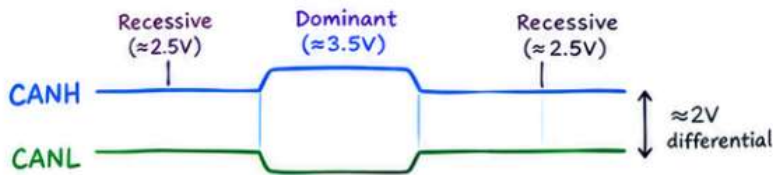
CAN BUS TOPOLOGY



CAN FRAME FORMAT



CAN SIGNAL WAVEFORM



BIT STUFFING

- If 5 consecutive bits are the same (0 or 1), the transmitter inserts an opposite bit (stuff bit).
- Helps in synchronization and error detection.
- Receiver removes the stuffed bit.



ERROR DETECTION MECHANISMS

- ✓ Bit stuffing error
- ✓ Form error
- ✓ CRC (Cyclic Redundancy Check) error
- ✓ Acknowledgment error
- ✓ Stuffing error



CAN ID TYPES

- 11-bit (Standard CAN)
- 29-bit (Extended CAN)
- Both are supported by CAN protocol.

TRANSMISSION MODES

- Normal mode - Regular operation
- Listen only - Receive only
- Loopback - Internal testing
- Silent - No transmission



ADVANTAGES

- ✓ High reliability
- ✓ Noise immunity
- ✓ Real-time communication
- ✓ Multi-node support



APPLICATIONS

- Automotive (ECU, sensors)
- Industrial automation
- Medical devices
- IoT systems



LIMITATIONS

- Lower data rate compared to UART / SPI
- Complex protocol
- Higher wiring cost



CONCLUSION : CAN is a robust, reliable and noise-resistant protocol, widely used for **real-time embedded communication**.



UART vs SPI vs I²C vs CAN

Quick Comparison of Embedded Communication Protocols ★

FEATURE	UART	SPI	I ² C	CAN
 Type	Asynchronous	Synchronous	Synchronous	Synchronous
 Wires (Connections)	2 (TX, RX)	4 (MOSI, MISO, SCLK, SS)	2 (SDA, SCL)	2 (CANH, CANL)
 Duplex	Full Duplex	Full Duplex	Half Duplex	Full Duplex
 Clock	No (Asynchronous)	Yes (Master provides)	Yes (Provided by SCL)	Yes (Synchronized on bus)
 Speed	Low to Medium (~115 kbps to a few Mbps)	High (Up to 50+ Mbps)	Low to Medium (100 kbps, 400 kbps, 1 Mbps, 3.4 Mbps)	Medium to High (125 kbps to 1 Mbps typical)
 Addressing	No Address (Point to Point)	No Address (Using SS line)	7-bit / 10-bit Addressing	11-bit / 29-bit Identifier (ID)
 Multi-device Support	No (Point to Point)	Yes (With multiple SS)	Yes (Multiple slaves)	Yes (Multi-master)
 Communication Distance	Short (~1-3 meters)	Very Short (On PCB)	Short (~1 meter)	Long (up to 1 km)
 Error Checking	No built-in (Need software)	No built-in (Need software)	No built-in (ACK/NACK only)	Yes (CRC, Error Frame)
 Use Case / Applications	Debug, GPS, Bluetooth Modules, PC Communication	Display, SD Card, Sensors, Flash Memory	Sensors, EEPROM, RTC, DAC, LCD Drivers	Automotive (ECU), Industrial, Robotics, Real-time Systems
 Topology	Point to Point	Star (With SS) Point to Point	Bus Topology	Bus Topology
 Cost	Very Low	Low	Very Low	Higher (Transceiver needed)

★ QUICK SUMMARY

UART

- Simple & easy
- 2 wires
- Good for debug & low speed



SPI

- Fast & efficient
- 4 wires
- Best for short distance devices



I²C

- 2 wires only
- Multiple devices
- Great for sensors & low speed



CAN

- Very reliable
- Long distance
- Best for industrial & automotive



WHEN TO USE?

Use UART when:

- You need simple serial communication
- Debugging (printf)
- GPS / Bluetooth modules
- PC to MCU communication



Use SPI when:

- You need high speed
- Short distance (on PCB)
- Display, SD Card, Flash memory, Sensors



Use I²C when:

- You need 2 wires only
- Multiple devices on bus
- Sensors, EEPROM, RTC, LCD, DAC



Use CAN when:

- You need high reliability
- Long distance
- Automotive, Industrial, Real-time systems



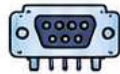
FINAL TIP : Understand the need, then select the protocol.
Good design = Right protocol + Right implementation!



INTERVIEW QUICK REVISION

UART vs SPI vs I²C vs CAN - Key Points & Interview Questions ★

1 UART



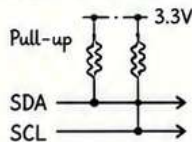
- ✓ Asynchronous communication
- ✓ 2 wires → TX, RX
- ✓ Full Duplex
- ✓ No clock line
- ✓ Start bit, Data bits, Parity (optional), Stop bit
- ✓ Simple & easy to implement
- ✓ Used for short distance communication
- ✓ Examples: Debug, GPS, Bluetooth, PC-MCU

2 SPI



- ✓ Synchronous communication
- ✓ 4 wires → MOSI, MISO, SCLK, SS
- ✓ Full Duplex
- ✓ Master-Slave architecture
- ✓ High speed (up to 50+ Mbps)
- ✓ No addressing (uses SS line to select slave)
- ✓ Best for short distance
- ✓ Examples: SD Card, Flash, Sensors, Display

3 I²C



- ✓ Synchronous communication
- ✓ 2 wires → SDA, SCL
- ✓ Half Duplex
- ✓ Master-Slave architecture
- ✓ Addressing (7-bit / 10-bit)
- ✓ Multiple slaves on same bus
- ✓ Speed: 100 kbps, 400 kbps, 1 Mbps, 3.4 Mbps
- ✓ Examples: EEPROM, RTC, Sensors, DAC, LCD

4 CAN



- ✓ Synchronous communication
- ✓ 2 wires → CANH, CANL (Differential)
- ✓ Multi-master architecture
- ✓ Message based with ID (11-bit / 29-bit)
- ✓ Long distance (up to 1 km)
- ✓ High reliability & error handling (CRC, ACK)
- ✓ Used in harsh industrial/automotive env.
- ✓ Examples: Automotive ECUs, Industrial Control, Robotics, Real-time Systems



MOST IMPORTANT INTERVIEW QUESTIONS

1 What is UART?

Asynchronous serial communication using TX and RX lines.



2 How does SPI work?

Synchronous communication using 4 wires. Data is transferred between master and slave.



3 What is I²C and its advantage?

Two wire protocol with addressing. Allows multiple slaves on the same bus.



4 What is CAN and why is it reliable?

Differential communication with error detection (CRC), ACK and arbitration. It is highly reliable in noisy environments.



5 What is the difference between UART and SPI?

UART ↔ SPI

UART is asynchronous (no clock) and slower. SPI is synchronous (uses clock) and faster.

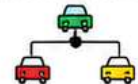
6 What is the difference between I²C and SPI?

I²C ↔ SPI

I²C uses 2 wires and supports multiple slaves with addressing. SPI uses 4 wires and is faster.

7 What is arbitration in CAN?

When multiple nodes transmit together, the message with lowest ID wins (non-destructive arbitration).



8 When would you choose which protocol?



UART : Simple serial communication
 SPI : High speed, short distance
 I²C : Multiple devices, short distance
 CAN : Long distance, high reliability

KEY TAKEAWAY

- ✓ Understand the need, then select the protocol.
- ✓ Consider Speed, Distance, Number of devices, Power & Complexity.
- ✓ Good design = Right protocol + Right implementation!



SAVE THIS POST
for quick revision
before your interview!



Good understanding of protocols = Strong Embedded Developer! 100